《软件体系结构与设计模式》三套模拟卷与深度解析

创建人：Soyaha 创建时间：2026.06.12 00:52

一、模拟卷A（基础巩固卷）

《软件体系结构与设计模式》期末考试模拟卷（A卷）
（闭卷考试 | 考试时间：120分钟 | 总分：100分）

一、判断题（本大题共5小题，每小题2分，共10分）

判断下列说法的正误，正确的在括号内打“√”，错误的打“×”。

1. （）设计模式的两大主题是“系统复用”与“系统扩展”。
2. （）在面向对象设计中，应优先使用类的继承关系，而不是对象组合关系来实现复用。
3. （）开闭原则的含义是：一个软件实体应当对扩展开放，对修改关闭。
4. （）对象适配器模式是对合成聚合复用原则的典型应用。
5. （）MVC架构模式融合了观察者、组合、策略等多种设计模式。

二、填空题（本大题共10空，每空2分，共20分）

请在下列各题的空白处填入正确的内容。

1. 面向对象七条核心设计原则包括：单一职责原则、开闭原则、里氏替换原则、依赖倒转原则、_____、_____和合成聚合复用原则。
2. 设计模式根据其目的可分为三类：创建型、**和**。
3. 工厂模式主要分为三种类型：简单工厂、**和**。
4. 适配器模式有两种实现方式：_____适配器（采用继承）和_____适配器（采用组合）。
5. 确保一个类仅有一个实例的模式是_____模式，其构造函数应被声明为_____。

三、单项选择题（本大题共15小题，每小题2分，共30分）

在每小题列出的四个备选项中只有一个是符合题目要求的，请将其代码填写在题后的括号内。

1. 下列哪一项不属于软件体系结构的三要素？（）
   A. 组件
   B. 连接器
   C. 协议
   D. 约束

2. 下列哪种内聚程度最高？（）
   A. 逻辑内聚
   B. 过程内聚
   C. 通信内聚
   D. 功能内聚

3. 以下关于管道-过滤器风格的描述，错误的是（）。
   A. 支持模块重用和并发处理
   B. 非常适合交互性强的应用系统
   C. 数据以流的形式经过一系列过滤器处理
   D. 编译器是这种风格的典型应用案例

4. “针对接口编程，不要针对实现编程”是下列哪项原则的核心思想？（）
   A. 单一职责原则
   B. 开闭原则
   C. 依赖倒转原则
   D. 接口隔离原则

5. “一个对象应当对其他对象保持最少的了解”，这句话描述的是（）。
   A. 合成聚合复用原则
   B. 迪米特法则
   C. 里氏替换原则
   D. 开闭原则

6. 在观察者模式中，当目标对象的状态发生改变时，它会（）。
   A. 调用观察者对象的方法来获取状态
   B. 自动通知所有已注册的观察者对象
   C. 将自己的引用传递给观察者
   D. 创建一个新的观察者实例

7. 以下模式中，主要意图是“将抽象部分与它的实现部分分离，使它们都可以独立地变化”的是（）。
   A. 适配器模式

B. 桥接模式
C. 装饰器模式
D. 代理模式

8. 以下哪项是策略模式（Strategy）的典型特点？（）
A. 定义一系列算法并封装，使它们可以相互替换
B. 允许对象在其内部状态改变时改变其行为
C. 将对象组合成树形结构以表示“部分-整体”层次
D. 为其他对象提供一种代理以控制对这个对象的访问

9. 下列设计模式中，属于创建型模式的是（）。
A. 组合模式（Composite）
B. 外观模式（Facade）
C. 原型模式（Prototype）
D. 访问者模式（Visitor）

10. 一个系统需要根据不同配置创建一套相互关联的产品对象（如不同风格的按钮、文本框），最合适的模式是（）。
A. 工厂方法模式
B. 抽象工厂模式
C. 建造者模式
D. 单例模式

（第11-15题基于以下类图或场景描述选择对应的设计模式）

11. 【类图识别】下图显示了一个抽象类`Component`，它有两个子类：`Leaf`和`Composite`。`Composite`对象可以聚合多个`Component`对象。这描述的是（）模式。
A. 装饰器
B. 组合
C. 桥接
D. 适配器

12. 【类图识别】一个`Target`接口，一个`Adapter`类实现了`Target`接口，并持有一个`Adaptee`类的实例。这描述的是（）。
A. 类适配器模式
B. 对象适配器模式
C. 装饰器模式
D. 代理模式

13. 【场景描述】在游戏中，整个场景中只应存在一个“Boss”角色，所有需要访问Boss的地方都引用这同一个实例。应使用（）模式实现。
A. 工厂方法
B. 原型

C. 单例
D. 建造者

14. 【场景描述】需要为一个已有的类`LegacyPrinter`增加一个新的接口`ModernPrinter`以适配新的客户端代码，且不希望修改原有类。应使用（ ）模式。
A. 装饰器
B. 代理
C. 适配器
D. 外观

15. 【场景描述】一个文档编辑器需要支持用户操作的撤销（Undo）与重做（Redo）。可以将每个用户操作封装成一个对象。这适合使用（ ）模式。
A. 命令（Command）
B. 备忘录（Memento）
C. 状态（State）
D. 策略（Strategy）

四、简答题（本大题共3小题，第1题8分，第2题10分，第3题12分，共30分）

1. **简述开闭原则（OCP）与依赖倒转原则（DIP）的含义，并说明它们之间的关系。**（8分）
2. **以下是某系统初期的一段设计代码：**

```
  class Rectangle {
     public double width;
     public double height;
}
class Square extends Rectangle {
     @Override
     public void setWidth(double width) {
          this.width = width;
          this.height = width; // 始终保持相等
     }
     @Override
     public void setHeight(double height) {
          this.height = height;
          this.width = height; // 始终保持相等
```

```
    }
}
```

问题：这段设计违反了哪条重要的面向对象设计原则？请解释原因。（10分）

3. **场景分析与模式选择**：一个天气预报站（`WeatherStation`）需要将实时气象数据（温度、湿度、气压）发布给多个显示设备，如当前状况显示（`CurrentConditionsDisplay`）、气象统计显示（`StatisticsDisplay`）和天气预报显示（`ForecastDisplay`）。当气象数据更新时，所有显示设备应能自动更新。
   - **（1）** 上述场景最适用于哪种设计模式？请写出该模式的名称。（2分）
   - **（2）** 请绘制该模式的简化UML类图（要求包含核心角色及其关系）。（6分）
   - **（3）** 简要说明在该场景中，如何体现“针对接口编程”的思想。（4分）

五、程序设计题（本大题共1小题，共10分）

请使用**单例模式（Singleton）** 实现一个线程安全的数据库连接池类 `DatabaseConnectionPool`。要求如下：

1. 确保整个程序中`DatabaseConnectionPool`只有一个实例。
2. 考虑多线程环境下创建实例的安全性。
3. 提供公共的静态方法 `getInstance()` 用于获取该唯一实例。

请在下方的代码框中写出 `DatabaseConnectionPool` 类的完整Java代码实现。

```
// 请在此处编写 DatabaseConnectionPool 类的代码
```

二、模拟卷B（进阶应用卷）

考试说明：本卷侧重设计模式的**综合应用、场景分析与复杂实现**。在掌握基本原则与模式识别的基础上，重点考查在复杂、多约束环境下，**选择、组合并实现**多种设计模式以解决实际工程问题的能力。满分100分，考试时间120分钟。

一、判断题（本大题共5小题，每小题2分，共10分）

（判断下列说法的正误，正确的打“√”，错误的打“×”。）

1. 【×】 在对象适配器模式中，适配器（Adapter）通过继承被适配者（Adaptee）来复用其功能，这是对合成聚合复用原则的典型应用。

> **解析**：对象适配器模式通过**持有（组合）** 被适配者的实例来工作，而非继承。文档明确指出“对象适配器模式是对合成聚合复用原则的典型应用”，其核心正是使用组合关系。

2. 【√】 MVC架构模式通常融合了观察者模式、组合模式和策略模式。

> **解析**：根据文档中对体系结构风格的分析，MVC（模型-视图-控制器）是典型的架构模式，其实现通常依赖于观察者模式（模型与视图间的通知）、组合模式（复杂用户界面的层次结构）和策略模式（控制器行为的可替换性）。

3. 【×】 访问者模式（Visitor）的核心优点是在不修改各元素类的前提下，为它们增加新的操作，因此它完全符合开闭原则。

> **解析**：访问者模式虽然能为元素类**增加新的操作**而不修改元素类本身，符合开闭原则对扩展开放的一面。但**当需要增加新的元素类时**，就需要修改访问者接口及其所有实现，这违反了开闭原则对修改关闭的要求。文档中指出开闭原则是总纲，但各模式有其适用的边界。

4. 【√】 桥接模式（Bridge）将抽象部分与实现部分分离，使它们可以独立变化，这有效降低了系统中多个变化维度间的耦合。

> **解析**：文档对桥接模式意图的描述为“将抽象部分与其实现部分分离，使它们可以独立地变化。用于处理多维度变化。”这直接对应了题目陈述。

5. 【×】 命令模式（Command）将请求封装为对象，其主要目的是为了实现请求的排队、日志记录或撤销操作，这与迪米特法则（最少知识原则）无直接关联。

> **解析**：命令模式通过引入“命令”对象作为中间层，使得请求的发起者（Invoker）与接收者（Receiver）解耦，发起者无需知道接收者的具体接口细节。这正体现了迪米特法则“只与直接朋友通信”的精神，降低了耦合度。

二、填空题（本大题共10空，每空2分，共20分）

1. 设计模式的三大分类是：**创建型**、**结构型**和**行为型**。
2. 在软件体系结构中，**管道-过滤器**风格属于数据流架构，其构件通过管道传递数据流，支持并发处理，但不适合**交互性强**的应用。
3. 面向对象设计中，“针对接口编程，而非针对实现编程”是**依赖倒置原则**的核心思想。而“一个类应该只有一个引起它变化的原因”描述的是**单一职责原则**。
4. 工厂方法模式定义一个创建对象的接口，但让**子类**决定实例化哪一个类，实现了对象创建的**延迟**。

5. 在状态模式中，状态转换的逻辑通常放置在**具体状态类（ConcreteState）** 中或**环境类（Context）** 中。前者更符合“将状态相关的行为局部化”的设计理念。
6. 组合模式使得客户端可以统一对待**单个对象**和**组合对象**，它常用于表示**树形（或部分-整体）** 结构。

三、单选题（本大题共15小题，每小题2分，共30分）

（从每题备选答案中选出一个正确答案。）

1. 下列哪项不是软件体系结构三要素？（**D**）
   A. 组件
   B. 连接器
   C. 约束
   D. 算法

> **解析**：文档明确指出软件体系结构的经典定义包含**组件、连接器、约束**三要素。

2. 某系统需要为一个复杂对象（如一台计算机）提供灵活的构建方式，允许相同的构建过程创建不同的表示（如游戏主机、办公电脑）。最适合采用（**B**）模式。
   A. 抽象工厂模式
   B. 建造者模式
   C. 原型模式
   D. 工厂方法模式

> **解析**：建造者模式的核心意图是“将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示”。抽象工厂关注产品族，工厂方法关注单一产品，原型关注克隆，均不直接匹配“构建过程固定，表示不同”的场景。

3. 观察以下简化的类图，图中最可能体现了哪种设计模式？（**C**）

```
[Client] --uses--> [Target]
                     ^
                     |
[Adapter] ---> [Adaptee]
```

   （箭头表示依赖或关联关系）
   A. 装饰器模式
   B. 桥接模式

C. 适配器模式（对象适配器）
D. 代理模式

> **解析**：该结构清晰地展示了适配器模式（对象适配器）的核心关系：Client使用Target接口，Adapter实现Target接口并持有（组合）一个Adaptee的实例。装饰器模式中装饰器与组件同接口且通常有继承关系；桥接模式抽象与实现分离；代理模式代理与真实主题同接口。

4. 关于MVC架构模式，下列说法错误的是（ **D** ）。
A. Model封装业务逻辑和数据状态
B. View负责向用户显示信息，并从Model获取数据
C. Controller处理用户输入，并更新Model或选择View
D. View可以直接修改Model的状态以保证数据同步

> **解析**：在标准的MVC中，View通常**观察（Observer）** Model的变化来更新自身，但**修改Model状态是Controller的职责**，以保持职责分离。View直接修改Model违反了这一分离原则。

5. 在下列场景中，最适合使用**策略模式**的是（ **A** ）。
A. 一个电商平台的计价系统，需要支持多种动态切换的优惠折扣算法（如满减、折扣券、会员价）。
B. 一个文档编辑器，需要实现复制、粘贴、撤销等操作的历史记录与回滚。
C. 一个企业级应用，需要为不同权限的用户（管理员、普通员工、访客）提供不同的数据访问界面。
D. 一个网络服务器，需要处理多种不同类型的客户端请求，并将请求分发给相应的处理器。

> **解析**：策略模式意图是“定义一系列算法，封装它们，并使它们可相互替换”。动态切换优惠算法是算法的替换，是策略模式的经典场景。B适合命令模式（支持撤销），C可能适合组合模式或基于状态的模式，D适合责任链模式。

6. 下列哪条设计原则被通俗地表述为“不要和陌生人说话”？（ **C** ）
A. 开闭原则
B. 接口隔离原则
C. 迪米特法则
D. 里氏替换原则

> **解析**：文档中明确将迪米特法则（Law of Demeter）描述为“最少知识原则”，其通俗表述正是“不要和陌生人说话”。

7. 在实现一个线程安全的单例时，采用“静态内部类”方式的主要优点是（ **B** ）。
A. 支持懒加载，且能防御反射攻击

B. 支持懒加载，且由JVM保证线程安全，实现简洁
C. 性能最优，因为实例在类加载时就创建
D. 可以方便地扩展为多例模式

> **解析**：静态内部类方式利用了JVM的类加载机制来保证初始化实例时的线程安全性，同时实现了懒加载（在调用`getInstance()`时才加载内部类），代码简洁高效。饿汉式是类加载时创建，懒汉式双重检查锁定代码稍复杂。

8. 某图形绘制系统有`Circle`、`Rectangle`等多种图形元素，现需要新增一个功能：计算所有图形元素的总面积。在不修改现有图形元素类代码的前提下，最适宜采用（**D**）模式。
A. 装饰器模式
B. 迭代器模式
C. 观察者模式
D. 访问者模式

> **解析**：访问者模式的核心意图是“在不改变各元素类的前提下，定义作用于这些元素的新操作”。新增“计算总面积”这一操作，正符合此场景。装饰器是为单个对象添加功能，迭代器是遍历，观察者是通知，均不直接匹配“为多个稳定类结构增加新操作”的需求。

9. 下列对于“合成/聚合复用原则”的理解，正确的是（**A**）。
A. 优先使用对象组合，可以使系统更灵活，降低类间耦合度。
B. 继承复用是“黑盒复用”，保持了较好的封装性。
C. 在桥接模式中，抽象化角色与实现化角色之间通常采用继承关系来连接。
D. 类适配器模式比对象适配器模式更符合此原则。

> **解析**：合成聚合复用原则的核心是“优先使用对象组合/聚合，而非继承”。组合是黑盒复用，耦合度低，更灵活。继承是白盒复用，破坏了封装性。桥接模式使用组合连接抽象和实现。类适配器使用继承，对象适配器使用组合，后者更符合此原则。

10. 关于设计模式，下列说法正确的是（**C**）。
A. 简单工厂模式符合开闭原则，因为增加新产品时无需修改工厂类。
B. 外观模式（Facade）通过为子系统引入新的、更复杂的接口来降低客户端的复杂度。
C. 中介者模式（Mediator）通过引入一个中介对象来封装一系列对象之间的交互，是迪米特法则的典型应用。
D. 原型模式（Prototype）主要用于创建成本高昂的对象，与对象复制无关。

> **解析**：A错，简单工厂增加新产品需修改工厂类的判断逻辑，违反开闭原则。B错，外观模式是**简化**接口，而非引入复杂接口。C对，中介者模式让对象通过

> 中介通信，减少了对象间的直接依赖，符合迪米特法则。D错，原型模式的核心正是通过复制（克隆）现有对象来创建新对象。

11. （**基于类图识别的进阶题**）分析以下类图结构，判断其最可能代表哪种模式？（**B**）

```
  [Component] <|— [Leaf]
[Component] <|— [Composite]
[Composite] o—> [Component] (children)
```

（`<|—`表示继承，`o—>`表示聚合）

A. 装饰器模式
B. 组合模式
C. 享元模式
D. 责任链模式

> **解析**：该图是组合模式的经典结构：`Component`是抽象组件，`Leaf`是叶子节点，`Composite`是复合节点，且`Composite`聚合了多个`Component`子对象，形成树形结构。

12. 在面向对象设计中，若一个基类的子类违反了基类定义的行为契约，导致在替换基类对象时程序出现错误，这主要违反了（**D**）。

A. 单一职责原则
B. 开闭原则
C. 依赖倒置原则
D. 里氏替换原则

> **解析**：里氏替换原则要求子类必须能够透明地替换其父类。子类改变父类的行为契约导致替换失败，是典型的违反LSP的情形。文档中以`Square`继承`Rectangle`导致`setWidth`和`setHeight`行为异常为例说明了这一点。

13. 一个文档解析系统，需要支持解析XML、JSON、CSV等多种格式的文档。针对每种格式，解析过程都包含“打开文档、读取数据、验证结构、关闭文档”等固定步骤，但每个步骤的具体实现不同。为设计这个系统，最好采用（**A**）模式来组织这些解析器。

A. 模板方法模式
B. 策略模式
C. 工厂方法模式
D. 状态模式

> **解析**：模板方法模式定义了“一个操作中的算法骨架，而将一些步骤延迟到子类中”。题目描述的“固定步骤，具体实现不同”正是模板方法模式的经典应用场景。策略模式侧重于整个算法的可替换性，而非定义算法骨架。

14. 以下关于体系结构风格的描述，错误的是（ **B** ）。

A. 层次结构风格支持基于抽象递增的设计，但可能带来层间转换的性能开销。

B. 管道-过滤器风格非常适合于需要高度交互性的图形用户界面应用程序。

C. 仓库风格以中央数据存储为核心，构件对其进行操作，适用于如数据库管理或语音识别等系统。

D. 事件驱动风格中，构件通过事件触发进行通信，降低了耦合度，但控制流难以跟踪。

> **解析**：文档明确指出管道-过滤器风格的**缺点**是“不适合交互性强的应用（倾向于批处理）”。因此B选项描述错误。A、C、D的描述均与文档中关于各风格特点的分析一致。

15. 在设计一个支持插件扩展的文本编辑器时，核心编辑器需要调用插件执行某些操作，但不应依赖具体插件的实现。为实现这一目标，最应该遵循（ **C** ）。

A. 接口隔离原则，为每种插件定义独立的微小接口。

B. 单一职责原则，确保编辑器核心和插件功能分离。

C. 依赖倒置原则，核心编辑器依赖抽象的插件接口。

D. 迪米特法则，限制编辑器对插件内部细节的了解。

> **解析**：问题的核心是“核心不依赖具体实现”，这正是依赖倒置原则（DIP）的要义：“高层模块不应该依赖低层模块，二者都应该依赖其抽象”。通过让编辑器依赖一个抽象的插件接口，具体插件实现该接口，即可达成目标。其他原则虽然相关，但DIP最直接地解决了所述依赖问题。

四、简答题（本大题共3小题，第1题8分，第2题10分，第3题12分，共30分）

1. **（8分）请结合具体实例，阐述开闭原则（OCP）与依赖倒置原则（DIP）之间的关系，并说明如何运用依赖倒置原则来实现开闭原则。**

> **参考答案**：
>
> - **关系阐述**：开闭原则（OCP）是目标，即软件实体应对扩展开放，对修改关闭。依赖倒置原则（DIP）是实现OCP的关键手段和核心机制。OCP告诉我们“做什么”（通过扩展来增加新功能），DIP则指导我们“怎么做”（通过依赖抽象而非具体实现来为扩展创造条件）。（2分）
> - **实例与运用**：
>
> i. **违反OCP与DIP的实例**：假设一个`OrderProcessor`类直接依赖一个`MySQLOrderRepository`类来保存订单。当需要支持`MongoDB`时，必须修改

`OrderProcessor`的代码，这违反了OCP，其根源在于违反了DIP（高层`OrderProcessor`依赖了低层具体类`MySQLOrderRepository`）。（2分）

ii. **运用DIP实现OCP**：

- **引入抽象**：定义一个`OrderRepository`接口，声明保存订单的方法。（1分）
- **依赖抽象**：修改`OrderProcessor`，使其仅依赖`OrderRepository`接口，而非任何具体实现类。（1分）
- **扩展实现**：创建`MySQLOrderRepository`和`MongoDBOrderRepository`类，分别实现`OrderRepository`接口。（1分）
- **达成OCP**：当未来需要新增其他数据库支持时，只需创建新的实现`OrderRepository`接口的类（如`PostgreSQLOrderRepository`），而**无需修改**`OrderProcessor`类的任何代码。系统对存储方式的扩展是开放的，同时对`OrderProcessor`的修改是关闭的。（1分）
  （实例可替换为文档中提到的“计价策略”、“数据库”等其他例子，原理相同）

2. **（10分）在开发一个智能家居控制系统时，你有多种设备（如灯、空调、窗帘），需要实现一个“离家模式”场景：一键关闭所有灯、关闭空调、关闭窗帘。现有设备类的接口各异（如`Light.off()`，`AirConditioner.turnOff()`，`Curtain.close()`）。**
   请回答：
   （1）为统一管理这些设备并执行“离家模式”，应选择哪种设计模式？说明理由。（4分）
   （2）绘制该模式的简化类图（需包含核心角色）。（4分）
   （3）简要描述在该场景下，该模式如何降低系统耦合度。（2分）

参考答案：

- **（1）模式选择与理由**：应选择**命令模式（Command）**。（2分）
  理由：命令模式将“请求”（如关闭设备）封装为一个独立的对象。在此场景中，可以将“关闭灯”、“关闭空调”、“关闭窗帘”分别封装为具体的命令对象。然后，可以轻松地将这些命令对象组合成一个“宏命令”（Macro Command）来执行“离家模式”。这使得：
  - 请求的发起者（如场景触发器）与请求的接收者（具体设备）解耦。
  - 易于支持命令的排队、日志记录和撤销（未来扩展）。
  - 能够动态地组合和配置命令序列。（2分）
- **（2）简化类图**：

```
        [Client] ---> [Invoker] (持有一个或多个Command)
                          |
                          v
                    [Command] <|— [ConcreteCommandA (Turn
                    [Command] <|— [ConcreteCommandB (Turn
                    [Command] <|— [ConcreteCommandC (Clos
                    [Command] <|— [MacroCommand] (聚合多个C
                          |
                          |  (持有引用)
                          v
                [ReceiverA (Light)]   [ReceiverB (AirCon
```

核心角色：Client（创建命令并设置接收者）、Invoker（调用命令执行）、Command接口、ConcreteCommand、Receiver。MacroCommand是ConcreteCommand的一种，它聚合并顺序执行其他命令。（4分）

- **（3）降低耦合度**：Invoker（场景执行器）只依赖于抽象的`Command`接口，完全不知道具体是哪个设备（Receiver）以及设备的具体接口。具体命令对象内部封装了与特定Receiver的交互细节。当设备接口变化或新增设备时，只需创建或修改对应的`ConcreteCommand`类，而Invoker和客户端其他部分无需改动，从而显著降低了各部分的耦合度。（2分）

3. **（12分）分析以下场景，并回答问题：**
   场景：一个图形编辑器，支持绘制多种图形（Shape），如圆形（Circle）、矩形（Rectangle）。现在需要新增两个功能：(a) 为所有图形添加一个“导出为SVG格式”的操作；(b) 未来可能会增加新的图形类型（如三角形）。
   现有代码结构：

```
 abstract class Shape {
    public abstract void draw();
    // 其他公共方法...
}
class Circle extends Shape { /* 实现draw方法 */ }
class Rectangle extends Shape { /* 实现draw方法 */ }
```

   方案一：在`Shape`抽象类中直接添加一个`exportToSVG()`抽象方法，迫使所有现有和未来的`Shape`子类实现该方法。
   方案二：采用访问者模式（Visitor），引入一个`ShapeVisitor`接口和`SVGExportVisitor`实现类。
   问题：

（1）分别指出方案一和方案二各自符合及违反了哪些重要的设计原则？（6分）
（2）从“开闭原则”和“单一职责原则”的角度，详细对比分析两个方案的优劣。（4分）
（3）如果未来图形类型非常稳定，但需要频繁增加对图形的新操作（如导出为PNG、计算周长、序列化为JSON等），哪种方案更优？为什么？（2分）

> 参考答案：
>
> - （1）原则符合与违反分析：
>   - 方案一：
>   - 违反：开闭原则（OCP）。当需要增加一个新操作（如导出PNG）时，必须修改`Shape`抽象类及其所有子类，对修改未关闭。（2分）
>   - 可能违反：单一职责原则（SRP）。`Shape`及其子类的职责从“表示图形并绘制”扩展到了“处理导出逻辑”，职责增加。（1分）
>   - 方案二（访问者模式）：
>   - 符合：开闭原则（OCP）。当需要为图形**增加新操作**（如新增`exportToPNG()`）时，只需新增一个具体的`Visitor`实现类（如`PNGExportVisitor`），而无需修改任何`Shape`类层次结构。图形类型对操作扩展是开放的。（2分）
>   - 违反：开闭原则（OCP）的另一面。当需要**增加新的图形类型**（如`Triangle`）时，必须修改`ShapeVisitor`接口及其所有实现类（如`SVGExportVisitor`），添加对`visit(Triangle)`的方法，这对“元素类型”的扩展未关闭。（1分）
> - （2）从OCP和SRP角度的优劣对比：
>   - 方案一：
>   - OCP角度：劣。无论是增加操作还是增加元素类型，都可能需要修改现有类，扩展性差。
>   - SRP角度：劣。`Shape`类的职责混杂了图形定义和多种业务操作，不符合单一职责。（2分）
>   - 方案二：
>   - OCP角度：优（对于操作扩展）。将操作与对象结构分离，新增操作容易，符合OCP对“操作扩展”开放的要求。但对于“元素类型扩展”不友好。
>   - SRP角度：优。`Shape`类层次只负责维护图形数据和行为（单一职责：表示图形），而各种导出操作的责任被分离到不同的`Visitor`类中（单一职责：执行特定操作），职责清晰。（2分）

- **（3）未来场景下的选择：**
  选择**方案二（访问者模式）**更优。（1分）
  理由：题目条件为“图形类型非常稳定，但需要频繁增加新操作”。访问者模式的优点正是**易于增加作用于稳定对象结构上的新操作**，而其缺点（难以增加新元素类型）在此条件下被规避。因此，在该场景下，访问者模式能更好地遵循开闭原则和单一职责原则。（1分）

五、程序设计题（本大题共1小题，共10分）

场景描述：你正在设计一个简易的文档解析器（Document Parser）框架。该框架需要处理多种类型的文档节点（Node），例如：文本节点（TextNode）、图片节点（ImageNode）、段落节点（ParagraphNode，可包含其他节点）。框架需要支持对文档节点树进行多种遍历和处理操作，例如：

- **操作A**：将整个文档节点树渲染（Render）为纯文本格式。
- **操作B**：统计文档中所有文本节点的总字符数。
- **未来可能增加的操作**：高亮显示特定关键词、生成文档大纲等。

要求：

1. 请选择并组合使用至少两种设计模式来设计该框架，以支持对稳定节点类型的**多种且易扩展的操作**。（提示：考虑节点类型的层次结构和操作的多样性）
2. 绘制出核心的UML类图，清晰地展示你所选模式的结构及它们如何协作。（5分）
3. 选择上述一种操作（A或B），用Java伪代码写出关键类的定义，并简要说明其执行流程。（5分）

参考答案：
1. 模式选择与组合理由：

- **组合模式（Composite）**：用于表示文档的树形节点结构。`ParagraphNode`作为复合节点，可以包含其他`Node`子节点；`TextNode`和`ImageNode`作为叶子节点。这使得客户端可以统一处理单个节点和节点树。（2分）
- **访问者模式（Visitor）**：用于封装对节点树的各种操作（渲染、统计等）。将每个操作（如`RenderVisitor`，`CountCharsVisitor`）实现为一个具体的访问者。这符合“节点类型稳定，操作频繁增加”的场景，便于扩展新操作而无需修改节点类。（2分）
  组合模式提供了稳定的对象结构（节点树），访问者模式则提供了在该结构上灵活扩展操作的机制，二者结合是处理此类问题的经典方案。（1分）

2. 核心UML类图：



说明：`Node`接口充当了访问者模式中的`Element`角色，声明`accept(Visitor)`方法。所有具体节点（包括叶子`TextNode`、`ImageNode`和复合节点`ParagraphNode`）实现此方法，通常实现为`v.visit(this)`。`Visitor`接口为每种节点类型声明一个`visit`方法。具体访问者实现这些方法来定义操作。复合节点`ParagraphNode`除了实现`accept`，还负责管理子节点列表。

3. 关键Java伪代码与执行流程（以操作B：统计字符数为例）：

```
  // 1. 节点接口（组合模式+访问者模式元素接口）
interface Node {
    void accept(NodeVisitor visitor);
}

// 2. 具体叶子节点
class TextNode implements Node {
    private String content;
    public TextNode(String content) { this.content = content; }
    public String getContent() { return content; }
    @Override
    public void accept(NodeVisitor visitor) {
        visitor.visit(this); // 将自身传递给访问者
    }
}
// ImageNode 类似，略

// 3. 具体复合节点
class ParagraphNode implements Node {
    private List<Node> children = new ArrayList<>();
```

```
    public void add(Node node) { children.add(node); }
    @Override
    public void accept(NodeVisitor visitor) {
        visitor.visit(this); // 处理自身（如果需要）
        for (Node child : children) {
            child.accept(visitor); // 关键：让访问者遍历所有子节点
        }
    }
}

// 4. 访问者接口
interface NodeVisitor {
    void visit(TextNode node);
    void visit(ImageNode node);
    void visit(ParagraphNode node);
}

// 5. 具体访问者：统计字符数
class CountCharsVisitor implements NodeVisitor {
    private int totalChars = 0;
    public int getTotalChars() { return totalChars; }

    @Override
    public void visit(TextNode node) {
        totalChars += node.getContent().length(); // 只统计文本节
    }

    @Override
    public void visit(ImageNode node) {
        // 图片节点无字符，不做累加
    }

    @Override
    public void visit(ParagraphNode node) {
        // 复合节点的字符数已由其子节点贡献，此处通常无需操作
        // 访问者通过 accept 方法已递归遍历了子节点
    }
}

// 6. 客户端使用流程
public class Client {
    public static void main(String[] args) {
        // 构建文档树
        ParagraphNode doc = new ParagraphNode();
        doc.add(new TextNode("Hello "));
        doc.add(new TextNode("World!"));

        // 创建访问者并应用于根节点
```

```
        CountCharsVisitor counter = new CountCharsVisitor();
        doc.accept(counter); // 触发整个树的遍历和统计

        // 获取结果
        System.out.println("总字符数: " + counter.getTotalChars(
    }
}
```

执行流程说明：

1. 客户端构建文档节点树（使用组合模式）。
2. 客户端创建具体的访问者对象（`CountCharsVisitor`）。
3. 客户端调用根节点（`ParagraphNode`）的`accept(counter)`方法。
4. 根节点的`accept`方法首先调用`counter.visit(this)`（处理自身，本例中无操作），然后遍历其所有子节点（`TextNode`实例），对每个子节点调用其`accept(counter)`方法。
5. 每个`TextNode`的`accept`方法调用`counter.visit(this)`，将自身（`TextNode`）传递给访问者。
6. 访问者`CountCharsVisitor`的`visit(TextNode node)`方法被执行，累加该文本节点的字符数到`totalChars`变量中。
7. 遍历结束后，客户端从访问者对象中获取统计结果。整个过程中，节点类不知道具体操作细节，操作类通过访问者接口与节点交互，实现了操作与结构的解耦。

三、模拟卷C（综合拔高卷）

（软件体系结构与设计模式）

考试时间：120分钟 总分：100分

考生须知：

1. 本试卷旨在考查学生对软件设计模式与原则的**综合应用、深度权衡及高级扩展能力**。
2. 试题内容基于历年真题核心考点，并在此基础上进行了**场景复杂化、模式复合化及实践深度化**的拔高设计。
3. 请将答案填写在答题卡指定位置，试卷上作答无效。

一、 综合判断题（本大题共5小题，每小题2分，共10分）

（请判断下列陈述是否正确，正确的打“√”，错误的打“×”）

1. **【设计模式权衡】** 在需要支持多种数据库的系统中，使用**抽象工厂模式**总是优于使用**工厂方法模式**，因为它能更好地保证产品族内对象的兼容性。 （）

> 📌 **考点**：工厂三型（抽象工厂 vs. 工厂方法）的适用场景与权衡。

2. **【体系结构与模式关联】** MVC架构风格中的 **“模型-视图”通知机制**，其底层最直接、最经典的设计模式实现是**观察者模式（Observer Pattern）**。 （）

> 📌 **考点**：宏观架构风格（MVC）与微观设计模式（Observer）的关联。

3. **【原则深度辨析】** **依赖倒转原则（DIP）** 要求高层模块不依赖低层模块，二者都应依赖抽象。这意味着任何两个有依赖关系的类之间都必须插入一个接口，否则就违反了该原则。 （）

> 📌 **考点**：对依赖倒转原则核心思想（面向抽象编程）的精确理解，避免教条化应用。

4. **【并发与模式】** 在Java中，使用**静态内部类**方式实现的单例模式，其线程安全性是由JVM的类加载机制保证的，且能实现懒加载，是《Effective Java》推荐的实现方式之一。 （）

> 📌 **考点**：单例模式的高级、线程安全实现方式及其原理。

5. **【模式复合应用】** 在实现一个支持撤销/重做（Undo/Redo）功能的图形编辑器时，**命令模式（Command Pattern）** 通常需要与**备忘录模式（Memento Pattern）** 结合使用，以保存和恢复对象的历史状态。 （）

> 📌 **考点**：行为型模式的组合应用场景（Command + Memento）。

二、 综合填空题（本大题共10空，每空1.5分，共15分）

1. 面向对象设计中，**“优先使用对象组合，而非类继承”** 是 _____ 原则的核心倡导。而**对象适配器模式**正是通过 _____ （填“继承”或“组合”）被适配者（Adaptee）的方式来工作的，是该原则的典型应用。

> 📌 **考点**：合成/聚合复用原则（CARP）及其在具体模式（适配器）中的体现。

2. 设计模式的两大主题是 _____ 与 _____。其中，创建型模式主要解决对象的 _____ 问题，使系统在“创建什么”、“谁来创建”、“何时创建”方面更具灵活性。

> 📌 **考点**：设计模式的根本目标与分类意图。

3. 在**访问者模式（Visitor Pattern）** 中，它适用于 `_____` **相对稳定，而** `_____` **易于变化**的系统结构。该模式将 `_____` 操作从对象结构中分离出来，使得新增操作变得容易。

> 📌 **考点**：访问者模式的适用场景、意图及与对象结构的关系。

4. **桥接模式（Bridge Pattern）** 的核心思想是将 `_____` 部分与 `_____` 部分分离，使它们都可以独立地变化。这种模式是使用 `_____` 关系来连接抽象和实现的典型代表。

> 📌 **考点**：桥接模式的定义、解决的问题及实现手段（组合关系）。

三、 深度选择题（本大题共15小题，每小题2分，共30分）

（每题只有一个正确答案，请将正确选项填入答题卡）

1. 【原则综合】以下关于设计原则的描述，**不正确**的是：
   A) **开闭原则（OCP）** 是面向对象设计的终极目标，**依赖倒转原则（DIP）** 是实现OCP的主要手段之一。
   B) **里氏替换原则（LSP）** 强调子类必须能够完全替换父类，这要求子类不能修改父类已定义的非抽象方法的行为约定。
   C) **接口隔离原则（ISP）** 主张为每个特定的客户端提供其专用的窄接口，这可能导致系统中接口数量急剧增加，在实际项目中应谨慎使用。
   D) **迪米特法则（LoD）** 要求一个对象只与其“朋友”通信，**外观模式（Facade）** 通过提供一个统一的接口来封装子系统，是遵循该法则的体现。

2. 【模式辨析】在一个文档编辑器中，需要为文本添加滚动条、边框和阴影等装饰功能，且这些功能可以动态地、透明地叠加。此时应优先采用：
   A) 适配器模式 (Adapter Pattern)
   B) 代理模式 (Proxy Pattern)
   C) 装饰器模式 (Decorator Pattern)
   D) 桥接模式 (Bridge Pattern)

3. 【模式综合】设计一个跨平台UI组件库，要求按钮（Button）、文本框（TextField）等组件能在Windows和macOS风格下呈现，且未来可能增加Android风格。组件类型与风格（平台）这两个维度都能独立扩展。最适合的模式是：
   A) 抽象工厂模式 (Abstract Factory Pattern)
   B) 建造者模式 (Builder Pattern)

C) 桥接模式 (Bridge Pattern)
D) 策略模式 (Strategy Pattern)

4. 【并发与模式】关于确保**单例模式**在多线程环境下的线程安全，以下说法错误的是：
A) **饿汉式**（静态初始化）利用类加载机制保证线程安全，但不能实现懒加载。
B) **懒汉式**（方法级同步`synchronized`）能实现懒加载，但每次获取实例都同步，性能有损耗。
C) **双重检查锁定（DCL）** 需对实例变量使用`volatile`关键字，以防止指令重排序问题。
D) 使用**枚举（Enum）** 实现单例是线程安全且能防止反射攻击的，但它同样无法实现懒加载。

5. 【体系结构与模式】在**管道-过滤器（Pipe-Filter）** 体系结构风格中，每个过滤器（Filter）作为独立处理单元，通过管道传递数据。从设计模式角度看，连接过滤器的“管道”最适合被抽象为：
A) 观察者模式 (Observer Pattern) 中的“主题（Subject）”
B) 责任链模式 (Chain of Responsibility Pattern) 中的“处理器链”
C) 中介者模式 (Mediator Pattern) 中的“中介者”
D) 迭代器模式 (Iterator Pattern) 中的“迭代器”

6. 【模式组合】实现一个可扩展的电子商务促销系统，基本折扣策略（如满减、打折）使用**策略模式**。现在需要新增一种“组合优惠”策略，该策略本身是由多个基本策略按规则组合而成的。为了优雅地实现“组合策略”，可以引入：
A) 装饰器模式 (Decorator Pattern)
B) 组合模式 (Composite Pattern)
C) 命令模式 (Command Pattern) 的宏命令
D) 访问者模式 (Visitor Pattern)

7. 【状态与策略辨析】一个网络连接对象有“已连接”、“监听中”、“已关闭”等状态，不同状态下其`send()`和`receive()`方法的行为不同，且状态间会依据特定事件（如收到数据包、用户指令）进行转换。最适合描述此设计的模式是：
A) 策略模式 (Strategy Pattern)
B) 状态模式 (State Pattern)
C) 命令模式 (Command Pattern)
D) 模板方法模式 (Template Method Pattern)

8. 【性能与模式】**享元模式（Flyweight Pattern）** 的主要目标是：
A) 提供一种方法顺序访问一个聚合对象中的各个元素，而又不暴露其内部表示。
B) 运用共享技术有效地支持大量细粒度对象的复用，减少内存开销。
C) 将对象的部分状态外部化，并可以在以后将对象恢复到这个状态。

D) 将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。

9. 【行为型模式综合】以下哪组模式都涉及对“请求”或“操作”的封装或参数化？
   A) 命令模式 (Command) 、策略模式 (Strategy) 、模板方法模式 (Template Method)
   B) 命令模式 (Command) 、责任链模式 (Chain of Responsibility) 、访问者模式 (Visitor)
   C) 观察者模式 (Observer) 、中介者模式 (Mediator) 、迭代器模式 (Iterator)
   D) 状态模式 (State) 、备忘录模式 (Memento) 、解释器模式 (Interpreter)

10. 【模式选择】系统需要为一个已有的、接口不符合新需求的类（ThirdPartyLogger）提供适配，但又不希望产生子类（因为ThirdPartyLogger是final类）。应选用：
    A) 类适配器模式 (使用继承)
    B) 对象适配器模式 (使用组合)
    C) 桥接模式
    D) 装饰器模式

11. 【原则应用】审查以下代码片段：

```
public class OrderProcessor {
    private MySQLDatabase db; // 直接依赖具体数据库实现
    public void saveOrder(Order o) {
        db.insert(o);
    }
}
```

    该设计违反了 `_________` 原则。重构时，应让`OrderProcessor`依赖于一个 `_________`。
    A) 开闭原则 (OCP), 具体数据库类
    B) 里氏替换原则 (LSP), 抽象数据库类
    C) 依赖倒转原则 (DIP), 抽象数据库接口
    D) 接口隔离原则 (ISP), 更小的专用接口

12. 【创建型模式对比】“构建一个包含房间、墙壁、门、窗的复杂房屋对象，且构建步骤必须清晰（打地基、砌墙、装门窗等），同时支持构建不同风格的房屋（如中式、欧式）。” 最适合此场景的模式是：
    A) 抽象工厂模式 (Abstract Factory Pattern)
    B) 建造者模式 (Builder Pattern)

C) 原型模式 (Prototype Pattern)
D) 工厂方法模式 (Factory Method Pattern)

13. 【模式结构识别】观察以下UML关系片段：`Client` --> `TargetInterface`，`AdapterClass` --|> `TargetInterface`， `AdapterClass` o-> `AdapteeClass`。这描述的是：
A) 装饰器模式 (Decorator Pattern)
B) 代理模式 (Proxy Pattern)
C) 对象适配器模式 (Adapter Pattern)
D) 桥接模式 (Bridge Pattern)

14. 【行为型模式应用】在一个事件驱动的GUI框架中，当按钮被点击、鼠标移动时，需要通知多个对此感兴趣的事件监听器执行相应操作。框架内部最可能采用的核心模式是：
A) 命令模式 (Command Pattern)
B) 观察者模式 (Observer Pattern)
C) 策略模式 (Strategy Pattern)
D) 访问者模式 (Visitor Pattern)

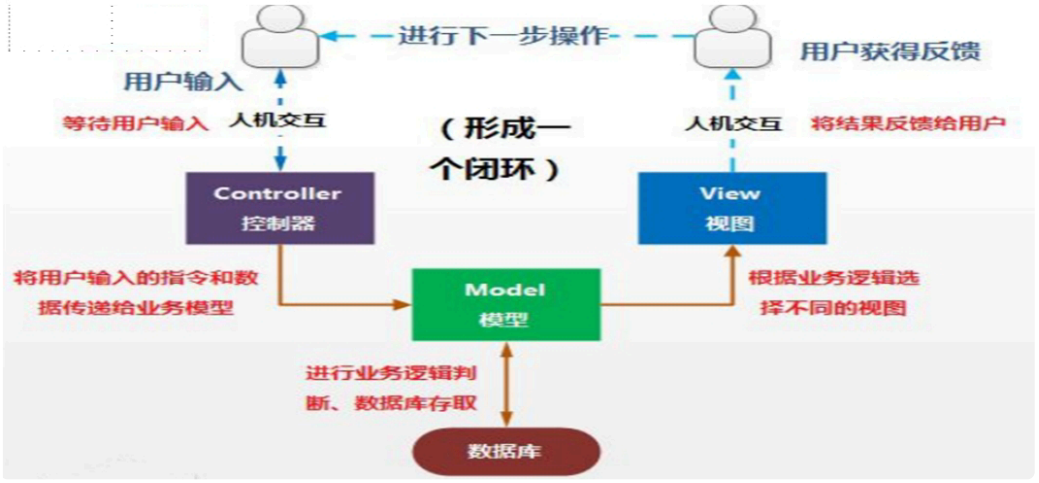
15. 【综合应用】设计一个支持**插件化**的文档处理工具。工具核心提供文档对象模型（DOM），第三方插件可以遍历DOM树，并对特定类型的节点（如文本节点、图片节点）执行自定义的分析或操作（如拼写检查、图片压缩）。为支持此功能，核心框架设计应优先考虑：
A) 组合模式 (Composite) + 迭代器模式 (Iterator)
B) 组合模式 (Composite) + 访问者模式 (Visitor)
C) 装饰器模式 (Decorator) + 策略模式 (Strategy)
D) 责任链模式 (Chain of Responsibility) + 命令模式 (Command)

四、 综合分析与简答题（本大题共3小题，共25分）

1. **(8分) 【原则重构与模式应用】**
场景：现有日志记录系统`Logger`，直接依赖具体的文件写入类`FileWriter`。现在需要增加支持数据库存储、网络发送等新的日志输出方式。
问题：
   - (a) 指出当前设计违反了哪个/哪些核心设计原则？（2分）
   - (b) 请运用相关设计原则进行重构，绘制重构后的核心类图（需包含抽象层、具体实现及客户端关系）。（4分）
   - (c) 说明你重构方案中应用了哪种设计模式，并简述该模式在此场景下的优势。（2分）

2. **(10分)【模式对比与选型】**
   场景：为“在线视频播放器”设计一个功能，根据用户的网络带宽（高速、中速、低速）动态切换视频的**播放清晰度**（1080P, 720P, 480P）。
   候选方案：有同学提出可用**策略模式（Strategy）**，也有同学认为用**状态模式（State）** 更合适。
   问题：
   - (a) 分别阐述使用**策略模式**和**状态模式**来实现该需求的**设计思路**（描述核心类及其职责）。（4分）
   - (b) 对比分析两种模式在此场景下的**主要区别**，并给出你认为更合适的模式选择及**理由**。（4分）
   - (c) 若未来需要增加根据“设备类型”（手机、电视）来切换**解码器**的功能，你的设计应如何扩展以优雅地处理“带宽”和“设备”这两个变化维度？（2分）

3. **(7分)【体系结构与模式集群】**
   背景：经典的**MVC（Model-View-Controller）** 架构风格被广泛应用于Web和桌面应用。



   问题：
   - (a) MVC风格中，**模型（Model）** 与**视图（View）** 之间的关系，通常由哪种设计模式来实现？简述该模式在此处的具体作用。（3分）
   - (b) 除了上述模式，MVC的实现通常还融合了**组合模式（Composite）** 和**策略模式（Strategy）**。请分别说明它们在MVC的哪个部分、起到了什么作用。（4分）

五、 系统设计与程序设计题（本大题共1小题，共20分）

【题目：可扩展的任务调度执行框架设计】

业务需求：需要设计一个**任务调度与执行框架**。该框架需要处理多种类型的任务（如“数据备份”、“报告生成”、“消息推送”），并满足以下核心需求：

1. **任务封装**：每个任务被封装成一个可执行的对象，支持记录日志、执行时间统计。
2. **执行策略**：框架支持不同的执行策略，例如：
   - **立即执行**
   - **定时执行**（在特定时间点执行一次）
   - **周期执行**（每隔固定时间重复执行）
3. **撤销与重做**：对于某些任务（如“数据库更新”），需要支持撤销（Undo）上一次执行的效果。
4. **任务组合**：支持将多个独立任务组合成一个“工作流”任务，按顺序批量执行。
5. **线程安全**：框架核心调度器在多线程环境下应是线程安全的单例。

设计要求：

1. **识别与选择模式**：分析上述需求，明确指出你准备采用哪些设计模式（至少3种）来分别应对需求1~4，并简述理由。（6分）
2. **绘制核心类图**：绘制该框架的核心设计类图。类图需清晰展示你选择的设计模式结构，以及各类（接口、抽象类、具体类）之间的关系（继承、实现、组合、依赖等）。（10分）
3. **关键代码片段**：选择框架中最能体现**某一种设计模式精髓**或**线程安全单例**的关键部分，用Java伪代码写出其核心结构。（4分）

提示：请仔细考虑模式之间的协作关系，力求设计一个内聚、松耦合、易于扩展的框架结构。

（试卷完）

内容由AI生成仅供参考